

基于端云协同与无屏交互的 智能眼镜 AI 助手的设计与实现

—— 面向 Rokid AI 眼镜的多场景语音智能体

应用平台：Android (minSdk 28 / targetSdk 33) + Rokid CXR 生态

硬件载体：Rokid Glasses 智能眼镜 + Android 手机 + HID 蓝牙遥控

摘 要

智能眼镜助手的核心问题并不在于选用何种模型，而在于如何让用户以最少的操作获取信息。用户佩戴眼镜后，双手通常处于被占用状态，视线也需要持续停留在当前场景，若仍要求其低头操作手机、切换应用，则与佩戴眼镜的初衷相背离。针对这一矛盾，本文基于 Rokid AI 眼镜实现了一套以语音为主的端云协同智能助手，将视觉识别、语言翻译、地图导航、运动感知与大模型对话等能力统一组织到同一条任务链路之中。

系统采用四层架构。设备连接层基于 CXR-M SDK 实现手机与眼镜的通信及近眼显示；基础能力层集成 Vosk 离线语音识别、CameraX 视觉采集、ML Kit 文字与二维码识别、TensorFlow Lite 目标检测以及 Android TTS 语音合成；智能理解层由本地规则匹配与 DeepSeek、智谱多模态模型共同构成；最上层为面向翻译、同传、导航、支付、无障碍、运动与解题等场景的应用层。端云之间遵循“隐私敏感与实时性要求高的任务在端侧完成，复杂对话与重算任务交由云端处理”的分工原则，从而保证网络不稳定乃至完全离线时，红绿灯识别、盲道识别、二维码识别、基础 OCR 与语音识别等核心能力仍然可用。

目前，眼镜连接、离线语音识别、红绿灯与盲道检测、地图导航及拍照解题等核心链路已完成联调验证，语音入口可对二十余个动作指令进行统一调度。本文按照需求分析、总体设计、详细实现与系统测试的顺序展开论述，并对已完成、待完善及后续补齐方向分别加以说明。

关键词：智能眼镜；端云协同；语音交互；近眼显示；目标检测；多模态大模型

Abstract

The core issue of a smart-glasses assistant lies not in which model is selected, but in how users can obtain information with minimal operations. When wearing glasses, users usually have their hands occupied and must keep their sight on the current scene; requiring them to look down at a phone and switch between applications contradicts the very purpose of wearing glasses. To address this problem, this thesis implements a voice-first, edge-cloud collaborative

assistant based on Rokid AI glasses, organizing visual recognition, language translation, map navigation, motion sensing and LLM-based dialogue into one unified task pipeline.

The system adopts a four-layer architecture. The device connection layer uses the CXR-M SDK for communication and near-eye display between the phone and the glasses. The capability layer integrates Vosk for offline speech recognition, CameraX for visual acquisition, ML Kit for text and QR-code recognition, TensorFlow Lite for object detection, and Android TTS for speech synthesis. The understanding layer combines local rule matching with DeepSeek and the Zhipu multimodal model. The top application layer serves translation, simultaneous interpretation, navigation, payment, accessibility, sports and problem-solving scenarios. The division between edge and cloud follows the principle that privacy-sensitive and real-time tasks are completed on the device, while complex dialogue and computation-heavy tasks are delegated to the cloud, so that core recognition capabilities remain available even under unstable or fully offline network conditions.

At present, the core pipelines of glasses connection, offline speech recognition, traffic-light and blind-path detection, map navigation and photo-based problem solving have been jointly verified, and more than twenty action commands can be uniformly scheduled through the voice entry. This thesis discusses requirement analysis, overall design, detailed implementation and system testing in sequence, and clarifies what has been accomplished, what remains to be improved, and how to fill the gaps.

Keywords: smart glasses; edge-cloud collaboration; voice interaction; near-eye display; object detection; multimodal LLM

目 录

1 绪论	6
1.1 研究背景与意义	6
1.2 国内外研究现状	6
1.3 主要工作与创新点	7
1.4 论文组织结构	8
2 相关技术	8
2.1 Rokid 眼镜与 CXR-M SDK	8
2.2 离线语音识别与合成	9
2.3 端侧视觉能力	9
2.4 云端大模型与多模态	9
2.5 地图导航服务	10
2.6 本章小结	10
3 系统需求分析与总体设计	10
3.1 需求分析	10
3.2 总体架构设计	12
3.3 端云协同分工策略	13
3.4 数据设计	14
3.5 本章小结	14
4 系统详细设计与实现	14
4.1 眼镜连接管理模块	14
4.2 语音交互闭环模块	16
4.3 视觉识别模块	19
4.4 AI 对话与多模态解题模块	22
4.5 地图导航模块	24
4.6 本章小结	26
5 系统测试与验证	26
5.1 测试环境	26
5.2 功能测试	26
5.3 性能测试方案	27
5.4 体验走查	28
5.5 本章小结	28
6 总结与展望	28
6.1 工作总结	28
6.2 存在的不足	29
6.3 未来展望	29

1 绪论

1.1 研究背景与意义

近年来，智能眼镜与大语言模型几乎同步进入快速发展阶段。Ray-Ban Meta 在拍摄与社交场景中取得了一定的市场声量，Google Glass 早期将"第一视角信息提示"的概念带入公众视野，XREAL 等厂商则在 AR 显示方向上持续探索。国内方面，Rokid 推出 Rokid Glasses，提供近眼显示、摄像头与麦克风等硬件能力，并通过 CXR-M SDK 与 YodaOS-Sprite 系统向开发者开放设备连接、自定义场景与拍照能力。眼镜具备看、听、显示的能力，但如何将这些能力在应用层加以组织，仍是尚未充分解决的问题。

回到日常使用场景，问题十分直观。骑行用户不愿频繁低头查看导航，视障用户不敢在过马路时将注意力交给手机屏幕，游客面对外语菜单时也不希望在拍照与翻译软件之间反复切换。手机配合蓝牙耳机解决了"听"的问题，却无法解决"看"的位置问题——二维码、路牌、红绿灯等信息最终仍需用户将手机举至眼前。眼镜的价值在于使信息停留在正常视线附近，将"看见"与"理解"两个动作重新合并。

从交互范式的角度分析，这里存在更深层的矛盾。手机交互建立在注视、触摸与层级页面之上，而头戴设备的用户始终处于运动之中，视线与双手均被占用，屏幕既不在眼前也不在指尖。将手机交互直接迁移到眼镜上并不可行，必须重新设计一套以语音为主、近眼显示为辅的交互方式。本研究的意义正在于此：验证头戴近眼显示场景下"语音优先、免手操作"的交互能否有效缩短用户完成任务的操作链，并探讨端云协同在离线场景下的能力边界。

1.2 国内外研究现状

1.2.1 智能眼镜产品与生态

国外产品普遍将重点放在社交分享与多模态问答上，眼镜更多被定位为手机生态的外设入口。Ray-Ban Meta 借助 Meta AI 提供拍照问答与实时翻译，但整体仍围绕手机生态运转；Google Glass 早期探索第一视角信息提示，后转向企业级应用；XREAL 则在

显示端持续优化，使 AR 画面更为轻薄。国内以 Rokid 为代表的生态采取开放路线，将硬件能力与 CXR-M SDK 一并开放，开发者可自行建立连接并定义交互场景，这为端到端的应用验证提供了条件。

1.2.2 端侧推理与多模态模型

技术底座在近两年进步明显。端侧推理方面，TensorFlow Lite 已能在手机上稳定运行 YOLO 系列检测模型，配合 int8 量化可将体积与功耗控制在较低水平；Vosk 等离线语音引擎实现了免网络的语音输入。云端方面，DeepSeek 在对话与意图理解上表现稳定，智谱 GLM-4V 等多模态模型支持图文结合的理解与推理。将各类能力进行组合并不困难，难点在于编排方式——何种任务置于端侧、何种任务交由云端、交互入口如何统一，这些才是智能眼镜应用需要解决的技术问题。

1.2.3 现有方案的主要不足

归纳已有产品与研究，主要存在三类不足。

- （1）部分产品仍以手机为中心，眼镜仅作为外接屏幕，未真正进入交互链路；
- （2）交互入口单一，或依赖触控，或依赖完全联网的语音，弱网环境下复杂识别直接失效；
- （3）翻译、导航、识别等功能相互独立，用户需在多个应用之间切换，能力彼此不联动。

针对上述不足，本文将眼镜作为交互链路中的真实节点，通过统一语音入口对多模态能力进行编排。

1.3 主要工作与创新点

本文完成的工作包括三个部分。第一，搭建了基于 Rokid AI 眼镜的端云协同助手应用，覆盖无障碍出行、跨语言沟通、导航、运动健康与拍照解题等场景；第二，实现了"语音输入—识别—意图判断—工具调用—结果表达"的统一调度框架，将二十余个动

作指令映射至对应能力模块；第三，逐一实现了红绿灯、盲道、二维码与 OCR 等端侧识别能力，并接入 DeepSeek 对话、智谱多模态解题与高德地图导航。

与已有方案相比，本文的差异集中在三点。

- （1）针对近眼显示场景定制了"语音+HID 遥控+抬头即看"的无屏交互方式，语音作为第一入口，遥控作为兜底，结果经 TTS 与近眼显示双通道返回；
- （2）关键识别能力全部端侧化，具备离线可用、低延迟以及避免原始画面外传等特性；
- （3）视觉、语音与地图多模态能力实现一体化融合，各模块分层解耦且可复用。

1.4 论文组织结构

论文共六章。第一章交代研究背景与现状，说明研究动因及已有工作；第二章介绍系统依赖的关键技术与选型取舍；第三章进行需求分析与总体设计，确定架构、端云分工与数据设计；第四章进入详细设计与实现，按模块介绍流程、关键类与核心代码；第五章给出测试与验证情况，包括功能用例与性能测试方案；第六章总结工作、分析不足并提出后续方向。

2 相关技术

2.1 Rokid 眼镜与 CXR-M SDK

Rokid Glasses 提供近眼显示、摄像头与麦克风，运行 YodaOS-Sprite 系统。CXR-M SDK（当前版本 1.0.3，仅提供 Android 版本）负责手机与眼镜的连接，支持数据通信、实时音视频获取与自定义场景交互。设备鉴权依赖保存在本地的 `sn.lc` 文件与 `CLIENT_SECRET`。SDK 提供的自定义视图机制允许开发者以 JSON 描述界面并下发至眼镜端渲染，这是"抬头即看"的底层能力。

选择 Rokid 而非封闭生态的原因在于其开放了连接与场景定制接口，可支撑完整的端到端应用验证。实际联调中发现，眼镜端与手机端的链路稳定性及字节流协议需要反复磨合，这也从侧面说明缺乏开放 SDK 时，第三方难以在头显设备上开发深度应用。

2.2 离线语音识别与合成

语音识别采用 Vosk，即基于 Kaldi 的离线识别引擎，支持中英文，识别过程不依赖网络。Vosk 由声学模型与语言模型构成，支持流式输入，可在说话过程中持续输出部分识别结果，有利于交互的流畅性。系统通过 AudioRecord 采集 16kHz 单声道 PCM 数据，再交由 Recognizer 进行流式识别。16kHz、单声道、16 位编码是语音识别领域通用的配置，兼顾精度与计算开销。

语音合成采用 Android 系统自带的 TextToSpeech。由于系统不具备语音活动检测能力，需要自行处理"播报与识别相互干扰"的问题。借助 UtteranceProgressListener 回调感知播报开始与结束，可实现在播报期间暂停录音、结束后恢复录音的协调机制。户外环境噪声较大，系统额外引入 WebRTC 降噪模块，对采集音频先降噪再识别。选择 Vosk 的理由在于语音入口必须免提、实时且不因断网而失效。

2.3 端侧视觉能力

视觉采集由 CameraX 负责，其同时支持 ImageAnalysis、ImageCapture 与 VideoCapture 三个 UseCase，可并行完成实时帧分析、拍照与录像，生命周期随界面自动管理。文字与二维码识别采用 ML Kit 的中文文字识别与条形码扫描模型，部署简单，适合实时摄像头分析。

红绿灯与盲道检测基于 TensorFlow Lite，在端侧加载 YOLOv8 目标检测模型。红绿灯使用 float32 版本（best_traffic_med_yolo_v8_float32.tflite）以优先保证识别精度，盲道使用 int8 量化版本（best_int8.tflite）以减小体积、提高速度。YOLOv8 为 anchor-free 检测器，输出 8400 个预测框及类别概率，需配合 Letterbox 预处理、置信度阈值、非极大值抑制（NMS）与连续帧稳定机制使用。选择端侧方案的原因在于此类识别要求低延迟，同时原始画面包含环境与行人信息，本地处理更为稳妥。

2.4 云端大模型与多模态

对话与意图分析由 DeepSeek 承担，通过 OkHttp 调用 deepseek-chat 对话补全接口。本地规则仅能覆盖高频指令，对于复杂表达需借助大模型的语言理解能力。意图分析过

程不上传原始音频，仅将识别文本发送至模型，由模型在预定义动作集合中选择动作名称，从而保证大模型负责语义理解、代码负责实际执行。

拍照解题采用智谱 GLM-4V 多模态模型，将当前画面编码后上传，使其同时理解图像与文字。此类任务对模型能力上限要求较高，对延迟的容忍度相对较高，因此置于云端。上传前会对图像进行压缩处理，仅传输完成任务所需的单张图像，以控制流量并减少数据外流。

2.5 地图导航服务

导航采用高德地图 SDK，包含 3D 地图、搜索与定位三个组件，可完成定位、POI 搜索、路径规划与 2D/3D 导航。定位综合 GPS、Wi-Fi 与基站信号，可覆盖室内外场景。调用时遵循高德隐私合规接口，先调用 `updatePrivacyShow` 与 `updatePrivacyAgree` 完成声明，再初始化地图。高德在国内出行场景具有成熟的地图数据与服务，可直接支撑完整的出行链路。

2.6 本章小结

选型思路可概括为：隐私敏感且要求实时的任务置于端侧，能力上限要求高且对延迟容忍的任务置于云端。语音与视觉能力对应不同的模型与工具，但最终均需接受统一语音入口的调度，这一约束直接影响第四章的模块设计。

3 系统需求分析与总体设计

3.1 需求分析

3.1.1 功能性需求

功能性需求从用户可直接感知的功能出发，整理为表 3-1。每个功能对应一类可由语音指令触发的动作，后续章节按表逐项实现。

表 3-1 功能性需求表

编号	功能需求	说明
FR1	眼镜连接管理	通过 CXR-M SDK 与蓝牙实现自动连接、断线重连与状态监测
FR2	语音交互	中英文离线语音识别与语音播报，构成免提闭环
FR3	意图理解与任务调度	将自然语言指令映射至翻译、导航、扫码、识别、运动等动作
FR4	视觉识别	红绿灯、盲道、二维码、中文 OCR 实时识别
FR5	AI 对话与解题	DeepSeek 对话、智谱多模态拍照解题
FR6	地图导航	定位、POI 搜索、路线规划与语音导航播报
FR7	运动健康	基于传感器实现步数统计与距离估算，展示健康状态
FR8	记录管理	基于 GreenDAO 的本地交互记录存储

3.1.2 非功能性需求

非功能性需求主要来自头戴设备的真实使用环境，语音指令从发出到收到结果的延迟需控制在可接受范围内，户外环境网络不稳定时识别能力不能完全依赖云端，摄像头与麦克风采集内容属于敏感信息，处理方式需谨慎。具体约束整理为表 3-2。

表 3-2 非功能性需求表

编号	非功能需求	说明
NFR1	离线可用	红绿灯、盲道、二维码、OCR、语音识别在无网条件下仍可用
NFR2	低延迟	端侧识别满足实时反馈，不依赖网络往返
NFR3	隐私安全	敏感识别端侧完成，云端最小上传，原始音频与画面不离设备
NFR4	功耗可控	计算任务合理分摊，避免重算全部集中于眼镜端
NFR5	可扩展	能力模块分层解耦，便于复用与新增场景

3.2 总体架构设计

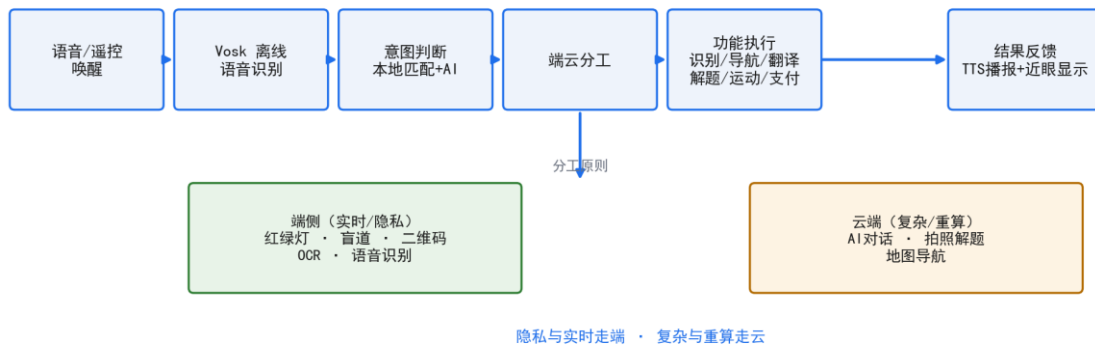


图 3-1 语音驱动的整体任务流程

系统采用自下而上的四层架构，整体流程如图 3-1 所示。设备连接层包含 CXR-M SDK、蓝牙 HID、CameraX 与 SensorManager，解决设备连接、画面采集与数据来源问题；基础能力层包含 Vosk、ML Kit、TensorFlow Lite 与 TTS，提供具体能力输出；智能理解层由本地规则、DeepSeek 与智谱多模态组成，负责理解用户意图；最上层为面向翻译、同传、导航、支付、无障碍、运动与解题等场景的应用层。

分层设计的主要收益在于能力复用。TTS 既为导航提供播报，也为红绿灯、盲道、翻译与 AI 对话提供播报；CameraX 一套采集可同时支撑预览、分析与录制；

GreenDAO 一套数据库可同时服务记录查询与业务数据。新增场景时大多只需对已有能力重新组合，无需重写交互逻辑。

用户说出指令后，先经 Vosk 识别为文字，再进行意图判断。高频命令走本地规则直接匹配，无法匹配的命令将文本交由 DeepSeek 从动作集合中选择。随后按照"隐私实时走端、复杂重算走云"的原则分流至具体模块，结果以语音与近眼显示两条通道返回。

3.3 端云协同分工策略

端云分工是系统的核心设计决策，各项能力按任务属性拆分，具体分配见表 3-3。

表 3-3 端云协同分工策略表

能力	端 / 云	分工理由
语音识别	端	免提输入要求低延迟与离线可用，原始音频含个人语音信息
红绿灯 / 盲道识别	端	实时提示要求低延迟，原始画面含环境与行人信息
OCR / 二维码	端	部署简单、实时分析，无需上传原始画面
语音播报	端	结果反馈通道，须即时
运动步数	端	本地传感器数据，无需上传
对话与意图分析	云	依赖大模型语言能力，只需上传识别文本
拍照解题	云	依赖多模态模型图文推理，仅上传当前题目图像
地图导航	云	依赖专业地图数据与服务

任务走端或走云的判定主要依据三个方面。

- (1) 延迟，识别类任务需在数百毫秒内反馈，云端往返无法满足；
- (2) 数据敏感度，原始画面与语音越敏感越应本地处理；
- (3) 能力上限，本地模型仅适用于固定场景，复杂对话必须依赖大模型。

三者综合即得到表中的分工。该分工同时带来鲁棒性收益：网络中断时，红绿灯、盲道、二维码、基础 OCR 与语音识别仍可维持核心体验，网络恢复后再启用云端对话与解题能力。

3.4 数据设计

数据存储采用 GreenDAO 本地数据库，用于保存交互记录与业务数据。交互记录表记录每次语音指令、识别动作、触发时间与结果状态，既服务于历史查询，也为后续个性化积累数据基础。整体遵循最小采集原则：云端仅接收完成任务所必需的数据，意图分析仅上传识别文本，解题仅上传当前单张题目图像，原始音频与连续画面默认不离开设备。

3.5 本章小结

需求、架构、分工与数据设计确定后，模块边界即已清晰。功能性需求表中的每一项均能在四层架构中找到对应位置，端云分工表给出各项能力的处理路径，数据设计约束信息的流转方式。第四章按模块展开实现，代码与设计一一对应。

4 系统详细设计与实现

4.1 眼镜连接管理模块

4.1.1 流程设计

连接模块处理"设备发现—蓝牙配对—CXR 鉴权—状态监测—断线重连"的完整链路。用户佩戴眼镜并打开应用后，系统通过 CXR-M SDK 与已配对眼镜建立蓝牙连接，读取本地 sn.lc 鉴权文件，配合 CLIENT_SECRET 完成设备鉴权。连接建立后可获取眼镜音量、亮度与电量等状态，并通过自定义视图机制向眼镜端下发界面。

断线重连是户外场景稳定性的关键。蓝牙连接易受距离与遮挡影响而中断，系统监听蓝牙状态回调，断开后自动进入重连流程，同时保留上一次的配网信息，避免用户反复手动配对。眼镜端状态亦实时同步至手机端，供界面展示与语音播报使用。

4.1.2 关键类与接口

连接逻辑收敛于 CxrUtil，主要接口包括：initBluetooth(context, device, callback) 初始化蓝牙连接，openCustomView(content) 下发自定义视图，setVolumeUpdateListener /

setBatteryLevelUpdateListener 监听设备状态，takeGlassPhoto(...) 调起眼镜拍照。自定义视图内容由一组 ViewModel 序列化为 JSON 后交由 CXR 渲染。

4.1.3 核心实现

连接初始化与自定义视图下发的核心代码如下。鉴权文件与密钥当前保存在客户端工程内，作为工程阶段的临时方案，正式发布前将迁移至服务端，详细说明见第七章合规讨论。

```
// 初始化蓝牙连接，连接成功后下发自定义视图
CxrApi.getInstance().initBluetooth(context, device, object : BluetoothStatusCallback {
    override fun onBluetoothStatus(isConnected: Boolean) {
        if (isConnected) {
            CxrApi.getInstance().openCustomView(Gson().toJson(customViewMap))
        }
    }
})

// 眼镜拍照能力调用
return CxrApi.getInstance().takeGlassPhoto(width, height, quality, photoPathResult)
```

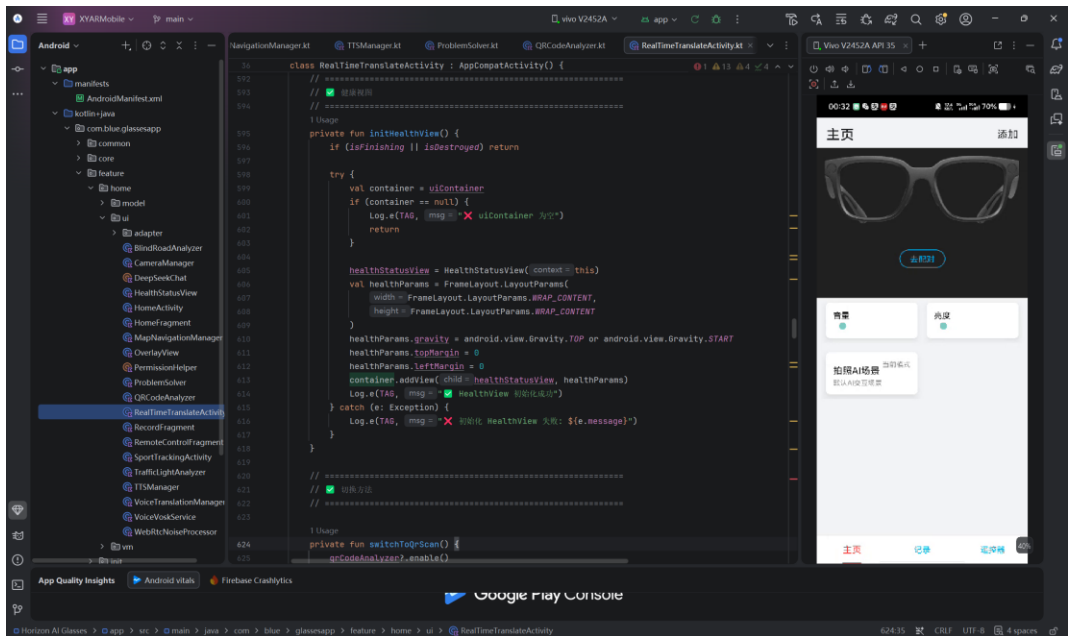


图 4-1 Android Studio 中的应用界面预览

4.2 语音交互闭环模块

4.2.1 流程设计

语音闭环是系统的入口，链路为"监听—采集与降噪—Vosk 识别—意图判断—工具调用—TTS 播报"。识别与播报之间存在协调逻辑：TTS 开始播报即暂停录音，播报结束恢复，防止系统播报占据麦克风。

监听线程中实现语音活动检测。程序以 RMS 计算每帧音频能量，超过阈值即判定为有人说话并重置静音计数器；低于阈值且静音持续超过设定时间，则判定一句话结束，取识别器最终结果。该机制保证持续监听时不会因空白音频频繁触发错误结果。

4.2.2 关键类与接口

VoiceVoskService 负责音频采集与 Vosk 识别，提供 startListening、pauseListening、resumeListening、switchToChinese、switchToEnglish、stopListening 等接口。中英文切换并非简单更换词表，而是释放当前 Recognizer、加载另一套声学模型，其间需预留资源释放时间以避免崩溃。TTSManager 负责播报，提供 speak 接口及 onSpeakStart / onSpeakDone 回调；VoiceTranslationManager 承担自然语言命令处理与意图分流。

4.2.3 核心实现

采集与静音超时处理是语音链路的关键部分，核心代码如下。需要说明的是，能量阈值与静音时长均为经验值，需在不同噪声环境下实测调优。

```
val bufferSize = AudioRecord.getMinBufferSize(16000, CHANNEL_IN_MONO, ENCODING_PCM_16BIT)
audioRecord = AudioRecord(CAMCORDER, 16000, CHANNEL_IN_MONO, ENCODING_PCM_16BIT,
    bufferSize * 2)
audioRecord?.startRecording()

// 识别线程：能量检测 → 降噪 → Vosk 流式识别
val rms = calculateRMS(buffer, bytesRead)
if (rms > 80.0) { // 检测到语音，重置静音计数
    silenceCount = 0
    lastSpeechTime = currentTime
} else if (silenceCount > SILENCE_FRAME_THRESHOLD
    && currentTime - lastSpeechTime > SPEECH_TIMEOUT_MS) {
    val result = recognizer?.result ?: "" // 静音超时，取最终结果
    resetRecognizer()
```

```

}

val processed = noiseProcessor.processAudio(buffer.copyOf(bytesRead))
if (recognizer?.acceptWaveForm(processed, processed.size) == true) {
    val text = parseResult(recognizer?.result ?: "")
    mainHandler.post { processResult(text) }
}

```

意图分流采用"本地快速匹配+AI 意图分析"两级策略。VoiceTranslationManager 维护指令映射表，对"运动""3D 导航"等高频命令直接本地匹配；无法匹配的命令将识别文本交由 DeepSeek，由模型从预定义动作集合中选择动作名称，动作集合与本地映射表保持一致。大模型仅负责语义理解，实际执行仍由代码控制，行为可控。

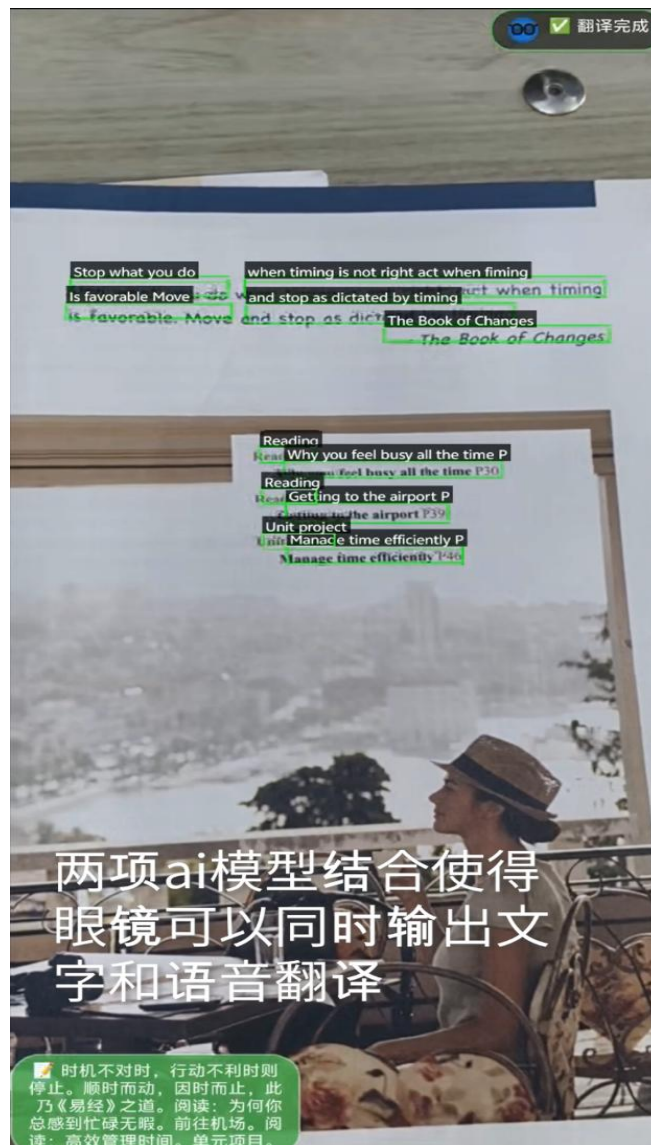


图 4-2 文字翻译与同传界面

4.3 视觉识别模块

视觉识别基于 CameraX 的 ImageAnalysis 持续取帧，端侧推理采用 TensorFlow Lite，红绿灯与盲道均配置连续帧稳定机制以抑制误检。三路能力共享同一取帧管线，仅挂载不同的 Analyzer，切换场景时替换分析器即可。

4.3.1 红绿灯识别

红绿灯识别模型为 best_traffic_med_yolo_v8_float32.tflite，输入尺寸 640×640，识别红、绿、黄三色。识别链路为：帧转 Bitmap → Letterbox 等比缩放居中填充 → 归一化 → 推理 → 解析 YOLO 输出 → 置信度阈值过滤 → NMS → 连续 3 帧稳定判定 → 回调播报，流程如图 4-3 所示。

```
private const val CONFIDENCE_THRESHOLD = 0.3f
private const val NMS_IOU_THRESHOLD = 0.5f
private const val STABILITY_FRAMES = 3

// 连续 3 帧结果一致才播报，瞬时抖动不播
stateBuffer.add(bestState)
if (stateBuffer.size > STABILITY_FRAMES) stateBuffer.removeAt(0)
val stable = stateBuffer.all { it == bestState }
if (stable && bestState != lastAnnouncedState) {
    lastAnnouncedState = bestState
    onResult(bestState, bestConf, detections)
}
```



连续帧稳定机制：3 帧结果一致才播报，抑制瞬时误检

图 4-3 红绿灯识别局部流程

实现中有两个细节值得展开。

(1) Letterbox 预处理：摄像头画面为 16:9 或 4:3，而模型输入为正方形，直接拉伸会破坏物体比例。程序先等比缩放，再于两侧补黑边，同时记录缩放系数与偏移量，推理出框后反变换回原图坐标，以保证识别框位置准确。

(2) 连续帧稳定：户外场景单帧误检较为常见，一帧红灯一帧绿灯交替会严重影响可用性。程序维护长度为 3 的状态队列，连续三帧结果一致才播报，可过滤大部分瞬时抖动。

4.3.2 盲道识别

盲道识别采用 int8 量化模型 best_int8.tflite，输出为 [1,6,8400]。系统检测盲道后计算其中心相对画面中心的水平偏离量 deviation，并据此生成方向提示，如"在盲道上"或"在左手 / 右手 X 米处"，同样经连续帧稳定后由 TTS 播报。偏离量换算为距离使用经验比例系数，方向提示仅作参考，不承诺精确距离。

```
// 取置信度最高的盲道检测框，计算偏离量
val deviation = (bestBoxCenterX - 0.5f) * 2f
val text = when {
    abs(deviation) < DEVIATION_THRESHOLD -> "在盲道上"
    deviation < 0 -> "检测到盲道，在左手 ${absDev * 2} 米处"
    else -> "检测到盲道，在右手 ${absDev * 2} 米处"
}
```



图 4-4 盲道识别实时画面

4.3.3 二维码与 OCR

二维码识别采用 ML Kit 条形码扫描，应用于扫码支付等场景；中文 OCR 采用 ML Kit 中文文字识别，供翻译链路提取文字。二者均在端侧执行，原始画面不传云端。OCR 输出为整页文字块，翻译链路先将其整理为可读文本再送云端翻译，避免直接上传整幅画面。

4.4 AI 对话与多模态解题模块

4.4.1 流程设计

普通对话通过 DeepSeek 完成，多模态解题将当前画面（相机帧或截图）编码后提交智谱 GLM-4V，使其同时理解图像与文字。解题前对画面进行压缩与裁剪，仅保留题目区域以减小上传体积。结果返回后，文字经 TTS 播报，涉及步骤推导的内容同步呈现于近眼显示。

4.4.2 核心实现

DeepSeekChat 通过 OkHttp 调用对话补全接口，模型固定为 deepseek-chat。不同任务采用不同温度参数：对话使用 0.7 以保留一定多样性，简单抽取使用 0.3 以保证确定性。意图分析场景调用 sendSimpleMessage，仅传入用户文本，由模型直接返回动作名称；对话场景则携带系统提示词，将"地平线"人设信息写入，保证回答风格一致。

```
val body = mapOf(
    "model" to "deepseek-chat",
    "messages" to listOf(
        mapOf("role" to "system", "content" to systemPrompt),
        mapOf("role" to "user", "content" to userMessage)
    ),
    "stream" to false,
    "temperature" to 0.7,
    "max_tokens" to 100
)
val request = Request.Builder()
    .url("https://api.deepseek.com/chat/completions")
    .addHeader("Authorization", "Bearer $API_KEY")
    .post(jsonBody.toRequestBody("application/json".toMediaType()))
    .build()
val response = withContext(Dispatchers.IO) { client.newCall(request).execute() }
```

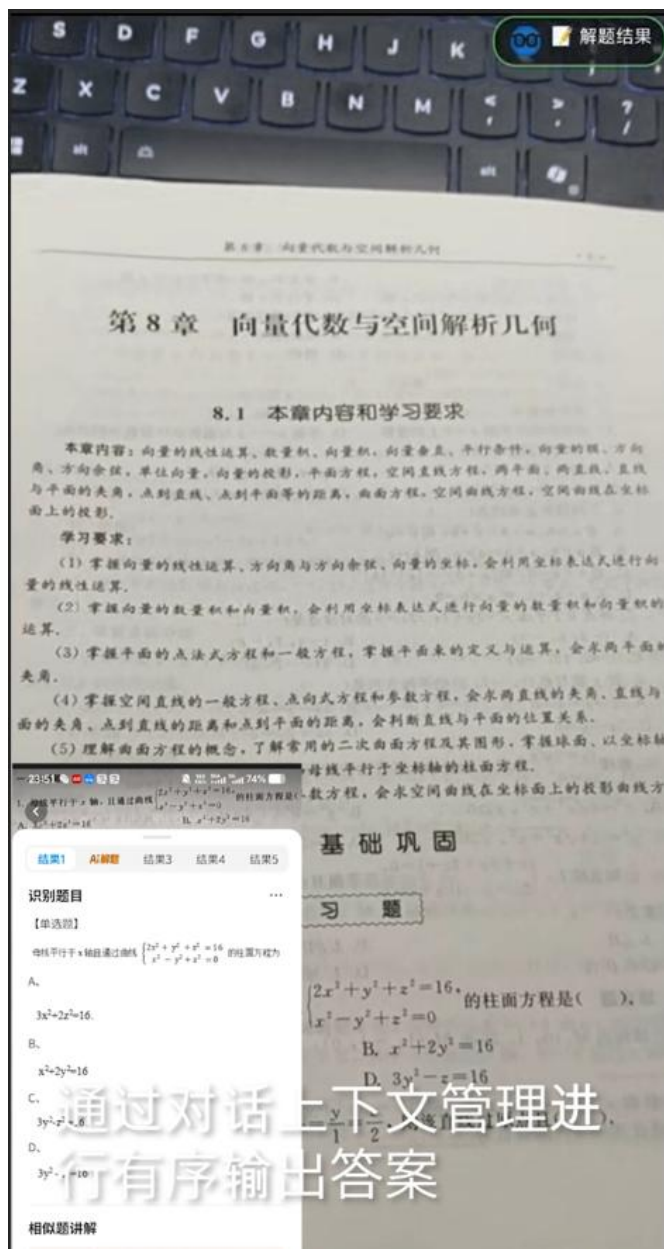


图 4-5 拍照解题实时画面

云端接口密钥目前直接写入客户端，对原型验证没有问题，但正式发布前必须迁移。解决方案是增加服务端代理，客户端仅请求自有服务器，由服务器持有密钥并转发至模型服务，避免密钥暴露于安装包内。

4.5 地图导航模块

4.5.1 流程设计

导航由 MapNavigationManager 统一管理，流程为"地图初始化与定位—目的地解析—POI 搜索—路径规划—路线绘制与语音播报"。导航可通过语音指令 `navigate / navigate_3d` 启动，也可通过 `cancel_navigation` 取消。目的地解析较为关键：用户说出"去西湖"后，系统需先将"西湖"解析为可检索关键词，再交由高德执行 POI 搜索，命中后才进入路径规划，而非将整句直接提交地图接口。

4.5.2 核心实现

地图初始化先调用高德隐私合规接口，再获取地图对象并启用定位；路径规划通过 RouteSearch 完成，结果以折线绘制于地图。语音播报在路线确定后触发，播报内容与距离、转向等实时状态绑定，经 TTS 统一出口输出，避免与识别抢麦。

```
// 隐私合规声明
AMapLocationClient.updatePrivacyShow(activity, true, true)
AMapLocationClient.updatePrivacyAgree(activity, true)

mapView.onCreate(savedInstanceState)
aMap = mapView.map
aMap.isMyLocationEnabled = true
aMap.moveCamera(CameraUpdateFactory.zoomTo(18f))

// 路径规划结果：将各 step 折线点收拢后绘制
val path = result.paths[0]
for (step in path.steps) {
    step.polyline?.forEach { allPoints.add(LatLng(it.latitude, it.longitude)) }
}
aMap.addPolyline(PolylineOptions().addAll(allPoints))
```

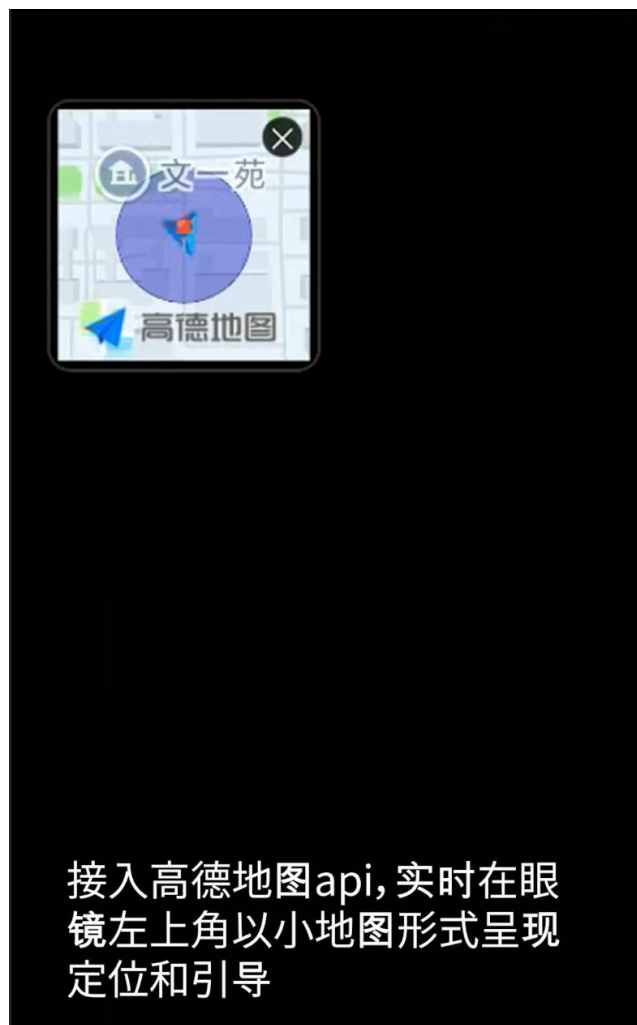


图 4-6 高德地图定位与路线界面

3D 导航与 2D 模式的区别在于视角与动画：3D 模式将相机视角调整为跟随车头方向，转动速度随转向调整，视觉上更接近实景导航。两种模式共享定位、路线与播报逻辑，仅地图相机配置不同，切换时只修改相机参数，不改变业务状态。

4.6 本章小结

五个模块实现后，"感知—理解—行动—表达"的闭环即告完整。语音与画面负责感知，本地规则与 DeepSeek 负责理解，各能力模块负责行动，TTS 与近眼显示负责表达。模块之间通过统一语音入口与动作集合松耦合，新增能力时只需在动作集合中增加一项并连接到对应模块。

5 系统测试与验证

5.1 测试环境

测试在真实的 Rokid Glasses 与 Android 智能手机组合上进行，手机为 Android 12 及以上、arm64 架构。端侧运行 YOLOv8 红绿灯模型（float32）、盲道模型（int8 量化）及 Vosk 中英文离线模型；云端依赖 DeepSeek deepseek-chat、智谱 GLM-4V 与高德地图 SDK。功能联调在开发机完成，弱网与断网场景在可控环境中模拟验证。

5.2 功能测试

核心链路已完成联调，主要功能用例及结果见表 5-1。测试以语音指令为入口，覆盖连接、识别、导航、翻译、支付、解题与运动等主要动作，每条用例均记录从发出指令到收到结果的完整过程。

表 5-1 功能测试用例表

编号	测试用例	预期结果	结果
TC1	语音"导航到 XX"	识别意图并进入导航链路	通过
TC2	语音"开启红绿灯识别"	启动端侧识别并语音提醒	通过
TC3	语音"开启盲道识别"	启动端侧识别并语音提示方向	通过
TC4	语音"翻译一下"	启动 OCR 与翻译	通过
TC5	语音"开启同声传译"	启动持续语音识别与翻译	通过
TC6	语音"扫码支付"	启动二维码识别	通过
TC7	语音"帮我看看这道题"	拍照并调用多模态解题	通过
TC8	语音"开始运动"	启动步数统计	通过
TC9	眼镜连接	CXR 鉴权与自定义视图下发	通过
TC10	TTS 播报与识别协调	播报期间暂停识别、结束恢复	通过

从测试结果看，语音入口可稳定触达各能力模块，红绿灯与盲道识别的连续帧稳定机制有效降低了瞬时误检，TTS 与识别的协调逻辑未出现抢麦现象。需要强调的是，

上述用例在演示环境下通过，并不代表所有环境均稳定，强噪声、强逆光、复杂路口等极端情况仍需进一步实测。

5.3 性能测试方案

性能指标用于为"端云协同"这一创新点提供量化依据，目前指标项与测量方法已经确定，具体数值将在小范围试点阶段实测填充，此处说明方法与目标。

端侧识别首帧延迟，记录从取帧到回调的时间差，目标为毫秒级实时反馈。语音识别准确率分中英文两组，在安静与噪声两种环境中以标准语料比对识别结果，重点考察 WebRTC 降噪后的效果。红绿灯与盲道误报率，通过连续识别统计误报次数，验证连续帧机制的有效性。在线与离线对比，在断网条件下逐项验证功能可用性，确认端侧能力维持核心体验。功耗与续航，按模块统计运行时耗电占比，重点考察相机持续分析与大模型调用的开销。

5.4 体验走查

功能联调之外，还对三条典型链路进行体验走查。无障碍出行链路考察红绿灯、盲道提醒的及时性，关注识别到播报的间隔以及误报对用户的干扰；跨语言链路考察翻译、同传的连续性，核心是翻译结果能否跟上对话节奏；出行链路考察导航的免手程度，统计从出发到到达过程中用户是否还需再次拿手机确认。

走查发现的问题集中在三处。一是强噪声下语音识别的鲁棒性仍有差距，需进一步调整降噪参数；二是连续播报时 TTS 队列的排队策略需优化，内容过长会挤占新的提醒；三是复杂路口的导航准确性依赖高德数据，个别场景需人工确认。上述问题均已列入后续优化清单。

5.5 本章小结

测试结论为：功能链路已验证通过，端侧能力在演示环境下表现稳定，定量性能指标待真实用户试点补齐。本系统更接近一个验证了可行性的工程原型，距离可量化的产品指标仍有差距，本章列出的指标即为下一步补齐的方向。

6 总结与展望

6.1 工作总结

本系统完成了多项工作。架构上建立了"设备连接—基础能力—智能理解—应用场景"四层结构，模块各自解耦且可复用；交互上打通了从一句话到结果反馈的完整语音闭环，二十余个动作指令均挂载于统一入口；能力上实现了端侧红绿灯、盲道、二维码、OCR 识别与云端 DeepSeek、智谱、高德服务的接入，遵循"隐私实时走端、复杂重算走云"的原则，离线可用性得到验证。

具体而言，红绿灯与盲道的连续帧稳定机制解决了单帧误检问题，使户外场景下的识别结果基本可信；语音与 TTS 的协调逻辑解决了"播报占用麦克风"的细节问题；意图分流的"本地匹配+AI 分析"两级策略在延迟与覆盖之间取得平衡。上述问题均来自实际联调中的具体环节。

6.2 存在的不足

系统仍存在若干不足。

- (1) 尚未实现真正的多步自主规划，任务调用顺序仍由代码固定，诸如"重新安排今日行程"等需求无法处理；
- (2) 长期记忆机制尚未建立，个性化能力有限；
- (3) 健康展示采用模拟数据，并非真实生理指标，仅能作为演示原型；
- (4) 云端接口密钥仍位于客户端，正式发布前需迁移至服务端代理或安全存储；
- (5) 部分视觉提示需将模型原始输出与用户可读提示更严格地绑定，避免识别结果与播报内容错位。

6.3 未来展望

后续工作大致分为四步。第一步将各能力统一封装为标准工具协议，由 Agent 决定工具调用顺序、失败重试与结果总结，推动系统从"多场景语音助手"演进为"个人智能体"。第二步建立长期记忆机制，使助手逐步掌握常用地点、常用语言与交互习惯，提

升回答与推荐的贴合度。第三步接入真实生理传感器，将健康展示升级为真实监测。第四步面向视障、旅行、运动三类人群开展小范围试点，实测性能指标，并评估 C 端订阅与 B 端垂直场景的商业化路径。

参考文献

[1] Rokid. CXR-M SDK 开发文档[EB/OL]. <https://custom.rokid.com/>. [2] Vosk API / Kaldi-based offline speech recognition toolkit[EB/OL]. <https://alphacephei.com/vosk/>. [3] Google. ML Kit Text Recognition (Chinese)[EB/OL]. <https://developers.google.com/ml-kit>. [4] TensorFlow. TensorFlow Lite 移动端机器学习推理框架 [EB/OL]. <https://www.tensorflow.org/lite>. [5] DeepSeek. DeepSeek API 文档 [EB/OL]. <https://platform.deepseek.com/>. [6] 智谱 AI. GLM-4V 多模态模型与开放平台 [EB/OL]. <https://open.bigmodel.cn/>. [7] 高德开放平台. Android 地图与搜索 SDK 文档[EB/OL]. <https://lbs.amap.com/>. [8] AndroidX. CameraX 生命周期感知相机库 [EB/OL]. <https://developer.android.com/training/camerax>. [9] greenrobot. GreenDAO Android 轻量级 ORM[EB/OL]. <https://greenrobot.org/greendao/>. [10] Ultralytics. YOLOv8: State-of-the-Art YOLO Models[EB/OL]. <https://docs.ultralytics.com/>.

附录 A 核心功能与代码模块说明

表 A-1 核心功能与代码模块说明

模块	功能说明
VoiceTranslationManager	核心语音任务入口：命令处理、本地快速匹配、AI 意图分析，调度翻译、同传、导航、扫码、识别、运动等能力
VoiceVoskService	AudioRecord 采集与 Vosk 识别，中英文模型切换、静音控制、WebRTC 降噪
TrafficLightAnalyzer	红绿灯端侧识别：TFLite 推理、置信度 / NMS / 连续帧稳定
BlindRoadAnalyzer	盲道端侧识别：量化模型推理、偏离度计算与方向提示
QRCodeAnalyzer	二维码检测（扫码支付等）
RealTimeTranslateActivity	CameraX、OCR 与多场景视觉功能的组织入口
MapNavigationManager	高德地图初始化、定位、POI 搜索、路线规划与 2D / 3D 导航
TTSTManager	中文语音初始化、播报队列、播报回调与异常处理
SportTrackingActivity	基于传感器的步数统计与距离估算
ProblemSolver	画面处理与智谱多模态提交，形成拍照解题闭环
DeepSeekChat	DeepSeek 接口通信，普通对话与意图分析
RemoteControlFragment	蓝牙 HID 与 CXR 辅助遥控输入
PermissionHelper	相机、麦克风、定位与媒体权限的场景化申请